Rover Mapping and Autonomous Exploration

ROS 2 Project Technical Report

Rover Mapping Project

November 1, 2026

Abstract

This report describes a ROS 2 rover simulation workspace that generates a procedural terrain, tracks rover position and battery state, publishes a local field of view (FOV), and selects movements in exploration or destination mode. The goal is placed inside a randomly generated target square and is disclosed to the explorer only when the goal cell appears in an FOV message. The project uses custom ROS 2 messages and services to connect its map, battery, and exploration nodes.

1 Project Overview

The project is a ROS 2 prototype for mapping and exploring a two-dimensional grid world. Each map cell has four environment values:

- height,
- roughness,
- muddiness, and
- wetness.

The simulated rover has integer grid coordinates and a floating-point battery level. It requests movement through a ROS service. The map node checks and charges the movement's battery cost before changing the rover position. A local FOV message provides the explorer with nearby terrain and target-area information.

1.1 Workspace packages

Package	Responsibility
`custom_msgs`	Defines the project's ROS messages and services.
`world_map`	Implements the map, battery, and explorer nodes, launch file, configuration, and switch-mode client.

2 System Architecture

2.1 Nodes

`map_node`
Generates the terrain and random coordinates, owns the rover's position, serves movement requests, publishes the local FOV, and displays terrain maps.

`battery_node`
Maintains battery state, publishes it periodically, and serves requests to deduct movement costs.

`explore_node`
Consumes FOV messages, maintains an observed-cell matrix and visualization, and chooses moves in exploration or destination mode.

`switch_mode_client`
Makes a one-shot request to switch the explorer between its two modes using a boolean ROS parameter.

2.2 ROS communication

Name	Type	Purpose
`/rover/fov`	Topic, `custom_msgs/msg/Fov`	Publishes rover pose, battery, target-square metadata, local terrain layers, and per-cell goal indicators.
`/rover/battery`	Topic, `std_msgs/msg/Float32`	Publishes current battery state.
`/rover/move`	Service, `custom_msgs/srv/Move`	Requests a grid displacement and battery deduction.
`/rover/update_battery`	Service, `custom_msgs/srv/Battery`	Accepts or rejects a battery deduction.
`/rover/switch_mode`	Service, `custom_msgs/srv/SwitchMode`	Changes the active explorer timer callback.

3 Terrain and Random World Setup

3.1 Terrain generation

The map node creates height, roughness, muddiness, and wetness grids with dimensions `length` by `width`. Height is initialized at the map center and expanded outward in increasing distance from that center. Each new height value uses already-initialized neighboring values and a random noise term. Roughness, muddiness, and wetness are independently sampled in the interval from 0 to 1.

The map node displays four terrain plots. The height plot also shows a star marker for the rover and updates it when an accepted move changes the rover position.

3.2 Coordinate generation

The map node generates coordinates after generating the terrain. The configurable `target_rad` determines a square target area with side length $2r + 1$, where $r =$ `target_rad`. The generated values are integers:

- **Goal** (g_x, g_y)**:** sampled from map coordinates.
- **Target center** (t_x, t_y)**:** selected so the goal lies within the target square. The complete square must fit inside the map.
- **Spawn** (s_x, s_y)**:** selected outside the target square. Its octile distance to the goal must be between $1.5r$ and $2r$, inclusive.

The current generator also requires the goal and spawn cells to be dry enough to walk on and the spawn to have at least one walkable adjacent cell. It makes up to 10,000 attempts and raises an error if it cannot find a valid configuration. On success, the map node logs the generated goal, target center, radius, and spawn.

The explorer does not receive the goal coordinates directly. The FOV message contains target-center and radius metadata, plus a boolean array indicating whether each visible FOV cell is the goal. The explorer records the goal only when that indicator appears in a received FOV.

4 Messages and Services

4.1 Custom message: `Fov`

The FOV message includes:

- rover coordinates `x` and `y`,
- battery level and FOV side length,
- target center and target radius,
- height, roughness, muddiness, and wetness arrays, and
- a boolean `goal_cells` array.

Each terrain layer and the goal indicator are serialized as a one-dimensional, row-major array. For FOV side length n, the arrays contain n^2 entries; row i, column j is at index $in + j$. The FOV timer publishes twice per second. Out-of-map terrain padding uses height 100.0 and -1.0 for roughness, muddiness, and wetness. Out-of-map cells are not marked as goals.

4.2 Custom services

`Move`
: Request fields are `delta_x`, `delta_y`, and floating-point `deduction`. The response contains `accepted` and `reason`.

`Battery`
: Accepts a floating-point `deduction` and returns `accepted` and `reason`. The battery node rejects a deduction that would make the balance negative; otherwise it subtracts the requested amount.

`SwitchMode`
: Accepts boolean `mode` and returns `success` and `message`. True selects exploration mode; false selects destination mode.

5 Movement and Battery Flow

For each movement request, the map node first waits for the battery service. It forwards the requested deduction to the battery node. If the battery node accepts the deduction, the map node applies `delta_x` and `delta_y` to the rover's coordinates and returns an accepted response. If the battery service is unavailable or rejects the deduction, the rover position is not updated.

The explorer sends movement requests asynchronously and waits for the updated FOV before planning another move. This prevents it from planning repeatedly against a stale rover pose.

6 Exploration and Destination Modes

6.1 Exploration mode

The explorer starts with `mode=True`. Its one-second timer calls `moveRequest`. The planner evaluates adjacent moves using local walkability, battery affordability, terrain values, and the number of newly observed cells. It uses a short lookahead when scoring moves and falls back to progress toward an unexplored frontier when the immediate information score does not select a move. Cells covered by an incoming FOV are marked explored in the explorer's boolean matrix.

6.2 Destination mode

The same one-second timer calls `destinationRequest` after the mode is switched to false. Before the goal is observed, destination planning is toward the target square and the planner can explore cells within that area. Once the goal indicator is observed, the planner selects walkable,

affordable steps that reduce octile distance to the discovered goal. It returns a zero-displacement result at the goal, and the timer callback then leaves the rover stationary.

The planner uses local FOV data and a one-cell movement neighborhood. It is a heuristic local planner, not a global path planner; obstacles or battery constraints can prevent it from finding a route even when a longer route exists.

6.3 Mode-switch client

The switch-mode client declares a boolean parameter named `mode`, whose default is false. It waits up to five seconds for the switch service and for the response, then logs the service result. The launch file currently starts this client with `mode=False`; therefore, the launched system switches to destination mode shortly after startup. Remove or comment out that launch action to leave the explorer in its default exploration mode.

7 Configuration and Running

The default configuration is in `src/world_map/config/rover.params.yaml`. It currently sets a 100×100 map, FOV side length 5, target radius 5, and maximum battery 100.0. Spawn and target coordinates are generated at runtime rather than loaded from this YAML file.

Build and launch from the workspace root:

```
colcon build --symlink-install
source install/setup.zsh
ros2 launch world_map map.launch.py
```

To switch modes manually while the system is running, invoke the client with the desired parameter:

```
ros2 run world_map switch_mode_client --ros-args -p mode:=true
ros2 run world_map switch_mode_client --ros-args -p mode:=false
```

The map and explored-state visualizations use Matplotlib. A graphical environment and an interactive Matplotlib backend are needed to see the plot windows.

8 Implementation Notes and Limitations

- The FOV arrays are flattened for ROS message transport; subscribers reshape them into $n \times n$ matrices using the advertised FOV size.
- The goal is hidden as a coordinate, but the target center and radius are sent in every FOV message so the explorer can search the target area.
- The current coordinate generator checks the local terrain condition of the goal, spawn, and at least one spawn neighbor. It does not prove global connectivity between spawn, target square, and goal.
- The explorer's observed matrix marks cells covered by the FOV. It therefore represents observed cells, not only rover-occupied cells.
- Destination movement is greedy with respect to locally visible walkable cells after goal discovery. It does not perform global pathfinding or guarantee recovery from a locally blocked route.
- The map and explorer receive map dimensions through ROS parameters. If dimensions are changed, the values configured for both nodes must remain consistent.

9 Project File Map

Path	Role
src/custom_msgs/msg/Fov.msg	Local terrain and goal-observation message.
src/custom_msgs/msg/Info.msg	Rover state message.
src/custom_msgs/srv/Move.srv	Rover movement request and response.
src/custom_msgs/srv/Battery.srv	Battery deduction request and response.
src/custom_msgs/srv/SwitchMode.srv	Explorer mode-switch request and response.
src/world_map/world_map/map_node.py	Terrain generation, movement service, FOV publisher, and map visualization.
src/world_map/world_map/battery_node.py	Battery publisher and deduction service.
src/world_map/world_map/explore_node.py	FOV consumer, exploration planner, destination planner, and explored-map display.
src/world_map/world_map/switch_mode_client.py	One-shot mode-switch client.
src/world_map/launch/map.launch.py	Launches the ROS nodes and configured mode-switch client.
src/world_map/config/rover.params.yaml	Default map, FOV, target-radius, and battery parameters.